C28 B2B Receivables Follow-Up Desk: AI Build Specification

Engineering build brief for the Fynd × NMIMS Hyderabad hackathon (17-18 Oct 2026). Give this whole document to your AI coding assistant before it writes any code.

1. How the AI must use this document

1. Read the whole document first. Then build in the order given in Section 13, one step at a time.
2. After each step, run the tests. Do not start the next step until the tests pass.
3. Do not add features that are not in this document. If something is unclear, choose the simplest option and write the assumption into `docs/assumptions.md`.
4. Inspect `Starter_Data/C28/fixture.json` before writing the loader. The field names in Section 4 are the expected meaning, not guaranteed spellings. Map the real names to the internal names in one place (`loader.py`) and never guess silently.
5. Never auto-send any message. Every follow-up is a draft that a human reviews.

Master checklist: Section 16 lists every required feature with its definition and acceptance check. Tick a feature only when its tests pass.

2. Problem in plain words

A small distributor sells goods to other businesses on credit. Customers pay later. The finance lead must decide who to chase, for how much, and how, without chasing a customer who has a valid dispute, has already paid, or has already promised to pay.

The tool must:

- Import invoices and receipts.
- Calculate the outstanding balance and how overdue each invoice is.
- Separate disputed money from collectable money.
- Record why a dispute happened and how it was resolved (replace goods, or settle financially).
- Explain in a sentence, for every invoice, exactly how its number was reached.
- Draft a reviewable follow-up plan.

- Save the result and the evidence.

3. Scope

In scope

- Load and validate `fixture.json` (and CSV if the fixture is CSV).
- Outstanding balance, partial receipts, unapplied receipts.
- Credit terms, due date, days overdue, ageing buckets, an as-of date control.
- Dispute amount, dispute cause, dispute resolution (replacement or credit), settled amount explanation.
- Promise-to-pay already received, including a broken promise (agreed date passed, not paid) that raises priority.
- Priority score and ranked list of collectable invoices.
- Template-based draft follow-up per customer.
- Export to JSON and CSV, evidence log in markdown.
- Unit tests for every business rule.

Out of scope (do not build)

- Sending emails, WhatsApp or SMS. Login, users, roles. A database. Payment gateways.
- LLM calls, machine learning, auto-matching of receipts to invoices.
- Multi-currency, tax, interest or late fee calculation.
- Any claim of recovered cash or production readiness.

4. Data model

4.1 Expected input (verify against the real fixture)

Invoice fields:

Internal name	Meaning
`invoice_id`	Unique id
`customer`	Customer name or id
`invoice_date`	Date issued (optional if the fixture has none)

Internal name	Meaning
`due_date`	Date payment is due
`amount`	Invoice total, a positive number
`as_of_date`	Date used for calculation (the fixture has one; the UI can override it)
`dispute_status`	`none`, `open`, `resolved` (map whatever the fixture uses)
`agreed_next_action`	Free text or code, for example a promise to pay

Receipt fields: `receipt_id`, `invoice_id` (empty means unapplied), `customer`, `amount`, `receipt_date` (optional).

4.2 Fields the tool adds for disputes

If the fixture has no such fields, add them as optional and default them to empty. They are edited in the UI and exported.

Field	Allowed values
`disputed_amount`	Number, 0 or more, never more than the balance before credit
`dispute_cause`	`damaged_goods`, `short_delivery`, `wrong_price`, `other`
`dispute_resolution`	`none`, `pending`, `replace_goods`, `credit_note`
`replacement_delivered`	true or false (used only when the resolution is `replace_goods`)
`credit_amount`	Money agreed off the invoice (used only when the resolution is `credit_note`)
`promise_date`	Date the customer promised to pay (optional)
`promise_amount`	Amount promised (defaults to the collectable amount; never more than it)

Field	Allowed values
promise_made_date	Date the promise was recorded (required when a promise exists; receipts from this date count toward the promise)

4.3 Money rules

- Use `Decimal` for all amounts. Round to 2 decimal places. Never use floats for money.
- Currency is a single unit (show ₹). No conversion.

5. Business rules (exact)

All rules are pure functions in `rules.py` and `disputes.py`. They take data and an `as_of` date and return values. They never read the clock or the UI.

5.1 As-of date

- Every calculation takes `as_of`. The default is the fixture's as-of date; the UI date picker overrides it.
- Changing `as_of` must change days overdue, buckets, priority and explanations everywhere at once, because nothing is stored.
- An invoice with `invoice_date > as_of` is not yet issued. Exclude it from the lists and show a note.
- A receipt with `receipt_date > as_of` is ignored for that calculation. If a receipt has no date, always count it.

5.2 Credit term and overdue

- `credit_days = due_date - invoice_date` in days (show it only if `invoice_date` exists).
- `days_overdue = max(0, as_of - due_date)`.
- If `as_of <= due_date`, the invoice is **Not due** and `days_overdue = 0`. On the due date itself it is not overdue; the day after, it is 1 day overdue.

5.3 Ageing buckets

Days overdue	Bucket
0	Not due
1-30	1-30
31-60	31-60
61-90	61-90
91 or more	90+

5.4 Outstanding balance

```
applied_receipts = sum of receipts linked to this invoice (counted per 5.1)
credit_applied   = credit_amount if resolution is credit_note, else 0
outstanding      = max(0, amount - applied_receipts - credit_applied)
```

- If `amount - applied_receipts - credit_applied` is negative, set `outstanding = 0` and flag the invoice as **Overpaid** with the excess amount.
- Receipts with no invoice id are **unapplied**. They are never subtracted from any invoice and never applied automatically. List them in a separate table by customer, and show a banner on that customer's invoices: "This customer has an unapplied receipt of X. Confirm before chasing."

5.5 Dispute split

The outstanding balance is split into a **review amount** (do not chase) and a **collectable amount** (can be chased).

Resolution	Review amount	Collectable amount	Meaning
`none`	0	outstanding	No dispute
`pending`	min(disputed_amount, outstanding)	outstanding - review	Dispute open. Chase only the undisputed part

Resolution	Review amount	Collectable amount	Meaning
`replace_goods`, not delivered	min(disputed_amount, outstanding)	outstanding - review	Waiting for the replacement. Disputed part held
`replace_goods`, delivered	0	outstanding	Replacement done. The money was never reduced, so the full balance is payable
`credit_note`	0	outstanding (already reduced by the credit)	Settled financially. Remaining disputed part (disputed_amount minus credit_amount) is payable

Extra constraints:

- `credit_amount <= disputed_amount`.
- `disputed_amount <= amount - applied_receipts` (it cannot exceed what was owed before credit).
- If the dispute is open and `disputed_amount` is empty, treat the whole outstanding balance as under review.

5.6 Invoice state

Each invoice gets exactly one state, checked in this order:

1. **Paid**: `outstanding == 0`. Drop from the priority list, show in a separate Paid tab.
2. **Under Review**: `collectable == 0` and `review > 0` (the whole balance is disputed).
3. **Promised**: an active promise exists (Section 5.6a) and `collectable > 0`. Do not chase until the promise date passes.
4. **Collectable**: `collectable > 0` and none of the above. A broken promise (Section 5.6a) lands here, flagged.

An invoice can be partly disputed. In that case its state is Collectable (or Promised) and it also shows a visible review amount, so disputed balances never disappear.

5.6a Promise-to-pay

A promise is recorded per invoice with `promise_made_date`, `promise_date` and `promise_amount` (Section 4.2). Only the latest promise per invoice is kept. Replacing a promise is noted in the evidence log.

```
promised_paid     = sum of receipts on this invoice dated from
promise_made_date to as_of
promise_shortfall = max(0, promise_amount - promised_paid)
```

Situation	Result
`as_of <= promise_date` and shortfall above 0	**Promised**: held, not chased, not ranked
shortfall is 0	**Promise kept**: no flag, normal rules apply
`as_of > promise_date` and shortfall above 0	**Promise broken**: state Collectable, flagged, with the note "Promise missed on {promise_date}, shortfall {promise_shortfall}"

- The promise date itself is still Promised. The day after, an unpaid promise is broken. Example: a customer who promised Monday and has not paid is Promised on Monday and broken on Tuesday.
- Receipts dated before `promise_made_date`, and receipts with no date, do not count toward the promise.
- A promise on a Paid invoice is ignored.
- A part payment keeps the promise broken for the shortfall only. The score uses `promise_shortfall / promise_amount` (Section 5.7).
- Changing `as_of` re-evaluates the promise like every other value. Nothing is stored.

5.7 Priority score

Only invoices in state **Collectable** (collectable amount above 0) are scored and ranked. Paid, Under Review, Promised and Not due invoices are excluded before scoring, and the disputed part of an invoice never adds to its score.

```
\text{priority} = 100 \times \left(0.45 \cdot \text{AgeScore} + 0.35 \cdot
\text{AmountScore} + 0.20 \cdot \text{BrokenPromise}\right)
```

Component	Weight	Definition	Why
AgeScore	0.45	`min(days_overdue, 90) / 90`	Older debt is harder to recover. The cap of 90 matches the 90+ bucket
AmountScore	0.35	`collectable_amount / largest collectable_amount among the ranked invoices`	Cash impact. Uses the collectable amount only, so disputed money never raises a rank
BrokenPromise	0.20	`promise_shortfall / promise_amount` when the promise is broken (Section 5.6a), else 0	A missed commitment signals higher risk. A part payment lowers it

- Keep the weights as constants at the top of `rules.py` (`W_AGE = 0.45`, `W_AMOUNT = 0.35`, `W_PROMISE = 0.20`, `AGE_CAP = 90`). They must sum to 1 (test 32).
- Sort by score descending, then `days_overdue` descending, then `collectable_amount` descending, then `invoice_id` ascending. Sort on the unrounded score and show it rounded to 1 decimal.
- If only one invoice is ranked, its AmountScore is 1. If no invoice has a collectable amount above 0, nothing is ranked.
- The weights are design choices, not an industry standard. List them under Assumed in the evidence log (Section 11).
- Every row carries a "why ranked here" sentence, for example: "Rank 1: score 60.0 = age 22.5 (45 days) + amount 17.5 (4000 of largest 8000) + broken promise 20.0 (shortfall 4000 of 4000)."
- Invoices that are Not due are listed in a separate "Not yet due" tab and are never ranked.

Worked example (largest collectable amount in the list is 8000):

- Invoice A: 45 days overdue, 4000 collectable, promised 4000 and paid nothing, so the promise is broken. Score = 100 × (0.45 × 0.5 + 0.35 × 0.5 + 0.20 × 1) = **60.0**.
- Invoice B: 80 days overdue, 1000 collectable, no promise. Score = 100 × (0.45 × 0.889 + 0.35 × 0.125) = **44.4**.
- A ranks above B. Without the broken promise, A would score 40.0 and rank below B.

5.8 Explanation sentence (required for every invoice)

Build it from parts. Only include a part when it applies.

Template: "Invoice {id}: amount {amount}. Receipts applied {applied}. {dispute sentence} {credit sentence} {promise sentence} Outstanding {outstanding}: collectable {collectable}, under review {review}. Due {due_date}, {days_overdue} days overdue ({bucket}) as of {as_of}. {score sentence}"

Dispute sentence: "Disputed {disputed_amount} ({cause in plain words})."

Resolution sentence by resolution:

- `pending`: "Dispute is open, so this part is held."
- `replace_goods` not delivered: "Agreed to replace the goods. Replacement not yet delivered, so this part is held."
- `replace_goods` delivered: "Goods replaced. Full amount remains payable."
- `credit_note`: "Settled financially: credit of {credit_amount} agreed. Remaining disputed amount {disputed_amount - credit_amount} is payable."

Promise sentence (only when a promise exists), one of: "Promised {promise_amount} by {promise_date}, received {promised_paid}. Promise active, so this invoice is held." or "Promised {promise_amount} by {promise_date}, received {promised_paid}. Promise kept." or "Promised {promise_amount} by {promise_date}, received {promised_paid}. Promise missed, shortfall {promise_shortfall}."

Score sentence (only for ranked invoices): "Score {score} = age {age_points} + amount {amount_points} + broken promise {promise_points}."

Worked example (amount 10000, receipts 4000, disputed 2000 for damaged goods, credit 1500):

> Invoice INV-1: amount 10000. Receipts applied 4000. Disputed 2000 (damaged goods). Settled financially: credit of 1500 agreed. Remaining disputed amount 500 is payable. Outstanding 4500: collectable 4500, under review 0. Due 2026-09-01, 25 days overdue (1-30) as of 2026-09-26.

Second example (broken promise: amount 4000, no receipts, promised 4000 by 2026-10-12, nothing received, as-of 2026-10-13, largest collectable amount in the list 8000):

> Invoice INV-2: amount 4000. Receipts applied 0. Promised 4000 by 2026-10-12, received 0. Promise missed, shortfall 4000. Outstanding 4000: collectable 4000, under review 0. Due 2026-08-29, 45 days overdue (31-60) as of 2026-10-13. Score 60.0 = age 22.5 + amount 17.5 + broken promise 20.0.

6. Validation rules

The loader validates before any calculation. It collects all errors, shows them in a table (row, field, problem), and does not crash. Valid rows still load; invalid rows are excluded and listed.

Check	Result
Missing required field (`invoice_id` , `customer` , `due_date` , `amount`)	Reject the row
`amount` is not a number or is 0 or negative	Reject the row
Date is not parseable	Reject the row
`due_date` earlier than `invoice_date`	Reject the row
Duplicate `invoice_id`	Reject the later row
Receipt `amount` not positive	Reject the receipt
Receipt points to an `invoice_id` that does not exist	Keep it as unapplied and flag "unknown invoice"
`disputed_amount` more than the balance before credit	Reject the dispute fields, keep the invoice
`credit_amount` more than `disputed_amount` , or `credit_note` with no `credit_amount`	Reject the dispute fields, keep the invoice
`replace_goods` or `credit_note` chosen with no `dispute_cause`	Reject the dispute fields, keep the invoice
Promise with a missing field, promise_date before promise_made_date, or promise_amount above the collectable amount	Reject the promise fields, keep the invoice

7. Project layout

```
c28-followup-desk/
  CONTEXT.md              (copy of Sections 1-6 for the AI)
  app.py                  (Streamlit UI only; no business rules)
  src/
    loader.py             (read file, map field names, validate)
    rules.py              (as_of, overdue, buckets, outstanding, state,
priority)
    disputes.py           (split, resolution rules, explanation text)
    drafts.py             (template follow-up text)
    export.py             (JSON, CSV, evidence log)
  tests/
    test_rules.py
    test_disputes.py
    test_loader.py
    test_drafts.py
  data/fixture.json       (copy of the starter data)
  output/                 (generated: starting_state.json, result.json,
result.csv, evidence_log.md)
  docs/assumptions.md
  requirements.txt        (streamlit, pandas, pytest)
  README.md               (how to run, how to test)
```

Stack: Python 3.10 or later, Streamlit, pandas, pytest. No other dependencies.

8. Module specifications

8.1 `loader.py`

- `load_fixture(path) -> (invoices, receipts, errors)`.
- Maps real field names to the internal names (one dictionary at the top).
- Converts dates to `datetime.date` and amounts to `Decimal`.
- Returns the errors list described in Section 6.
- Also returns the fixture's own `as_of_date` for the UI default.

8.2 `rules.py`

- `days_overdue(due_date, as_of) -> int`
- `bucket(days) -> str`

- `applied_receipts(invoice, receipts, as_of) -> Decimal`
- `outstanding(invoice, receipts, as_of) -> Decimal`
- `unapplied_receipts(receipts, as_of) -> list`
- `promise_status(invoice, receipts, as_of) -> (status, promised_paid, shortfall)` (Section 5.6a)
- `invoice_state(...) -> str`
- `priority_score(invoice_eval, max_collectable) -> (score, age_points, amount_points, promise_points)` (Section 5.7)
- `rank_invoices(evaluated_invoices) -> list` (Section 5.7)

8.3 `disputes.py`

- `split_balance(invoice, outstanding) -> (review, collectable)` (Section 5.5)
- `validate_dispute(invoice, applied) -> list[str]` (Section 6 dispute checks)
- `explain(invoice_eval) -> str` (Section 5.8)
- Plain-language cause names: `damaged_goods` is "damaged goods", `short_delivery` is "short delivery", `wrong_price` is "wrong price", `other` is "other reason".

8.4 `drafts.py`

- `draft_for_customer(customer, invoice_evals, as_of) -> dict` with keys `customer`, `status` (always `"DRAFT - not sent"`), `subject`, `body`.
- Uses only templates (Section 10). No external calls.

8.5 `export.py`

- `write_result_json(evals, path)`: one record per invoice with every computed field, the state, the explanation, and the as-of date used.
- `write_result_csv(evals, path)`: the same fields, flat.
- `write_evidence_log(path, ...)`: Section 11.

A single function `evaluate_all(invoices, receipts, as_of) -> list[InvoiceEval]` in `rules.py` runs everything in order and is the only thing the UI calls for numbers.

9. UI specification (Streamlit, one page)

Sidebar

- Load file (default `data/fixture.json`, or an upload).
- As-of date picker (default from the fixture).
- Customer filter.
- Show validation errors, if any.

Top strip

- Four numbers: total outstanding, collectable, under review, promised.

Tabs

1. **Priority list**: ranked Collectable invoices with columns for rank, customer, invoice, due date, days overdue, bucket, collectable, review, score, promise status, "why ranked here".
2. **Under review**: invoices or parts held for disputes, with cause and resolution.
3. **Promised**: invoices waiting on a promise, with promise date, promised amount, received so far and shortfall.
4. **Not yet due** and **Paid**.
5. **Unapplied receipts**.
6. **Draft plan**: one draft per customer (Section 10), editable text box, a "Mark reviewed" checkbox. No send button.

Invoice detail panel (opens when a row is chosen)

- The explanation sentence from Section 5.8.
- A dispute form: disputed amount, cause, resolution, replacement delivered (shown only for `replace_goods`), credit amount (shown only for `credit_note`), promise made date, promise date, promise amount. A Save button runs `validate_dispute`; on failure it shows the problem and does not save.
- A short audit table: amount, receipts, credit, outstanding, review, collectable, state.

Footer

- Export button: writes `output/result.json`, `output/result.csv` and updates `output/evidence_log.md`.

All numbers on screen come from `evaluate_all`. The UI contains no calculation logic.

Dispute edits live in session state and are written into the exported result. On first load, save the untouched data once to `output/starting_state.json`.

10. Draft follow-up templates

One draft per customer, covering all that customer's Collectable invoices. Choose the tone from the oldest Collectable invoice's bucket. A broken promise raises the tone to at least Firm (see the last row).

Situation	Tone	Template
Oldest bucket 1-30	Gentle	"Hello {customer}, a friendly reminder that invoice(s) {ids} totalling {amount} passed the due date. Could you please share the payment date?"
Oldest bucket 31-60	Firm	"Hello {customer}, invoice(s) {ids} totalling {amount} are now {days} days overdue. Please confirm payment by {suggested date}."
Oldest bucket 61 or more	Formal	"Dear {customer}, invoice(s) {ids} totalling {amount} are over {days} days overdue. Please arrange payment or contact us to discuss this week."
Customer has Under Review amounts	Add a line	"We are separately reviewing {review_amount} on invoice {id} and are not asking for that amount now."
Only Promised or Not due invoices	No chase	"No message needed. Next check: {promise_date}."
Only Under Review invoices	Dispute note	"We are looking into your concern about invoice {id}. We will update you on the resolution."
Collectable invoice with a broken promise	Firm or stricter, never Gentle	"Hello {customer}, you had agreed to pay {shortfall} for invoice {id} by {promise_date}. We have not received it yet. Please confirm a new payment date."

Rules:

- Amounts in a draft use the collectable amount, never the full outstanding balance.
- Every draft shows status "DRAFT - not sent".
- The user may edit the text and tick "Mark reviewed". Nothing is sent.

11. Export and evidence

`result.json` and `result.csv` carry the as-of date used and every field per invoice, so the result can be reproduced.

`evidence_log.md` has four sections:

1. **Starting state**: the original behaviour (an overdue list with no dispute handling), the fixture file name, and the number of invoices and receipts loaded.
2. **Negotiated scope**: add disputes (cause and resolution) and partial receipts to collection priority. Write the final agreed scope here, in the team's words.
3. **Final evidence**: test results (passed out of total), counts by state, and the validation errors found in the fixture.
4. **Observed versus assumed**: a two-column table. Observed means the tool produced it from the fixture. Assumed means anything not shown by the data (for example, "credit days are standard"). The priority weights (0.45 age, 0.35 amount, 0.20 broken promise), the 90-day age cap and the amount normalisation rule are also Assumed: they are design choices, not an industry standard. Include the statement: "Figures come from synthetic data. No cash recovery or production readiness is claimed."

12. Tests (write these before the UI)

Use pytest. Each row is a test. Dates are examples; keep them in the tests as constants.

#	Case	Expected
1	Invoice 10000, receipts 4000	outstanding 6000
2	Invoice 10000, receipts 10000	outstanding 0, state Paid, excluded from the priority list
3	Receipts 12000 on a 10000 invoice	outstanding 0, flagged Overpaid by 2000
4	Due 2026-09-01, as-of 2026-09-26	25 days, bucket 1-30

#	Case	Expected
5	Same invoice, as-of 2026-10-02	31 days, bucket 31-60
6	as-of equals due date	0 days, Not due
7	as-of before due date	0 days, Not due, not ranked
8	Days overdue 30, 31, 60, 61, 90, 91	Buckets 1-30, 31-60, 31-60, 61-90, 61-90, 90+
9	Outstanding 6000, disputed 2000, pending	review 2000, collectable 4000, state Collectable, disputed part visible
10	Whole 6000 disputed, pending	review 6000, collectable 0, state Under Review
11	`replace_goods`, not delivered, disputed 2000	review 2000, collectable the rest
12	Same, then delivered	review 0, collectable 6000
13	`credit_note`, disputed 2000, credit 1500, balance before credit 6000	outstanding 4500, review 0, collectable 4500
14	Credit larger than disputed	validation error, dispute fields rejected
15	Disputed larger than the balance	validation error
16	Unapplied receipt (no invoice id)	not subtracted from any invoice, listed separately with a customer banner
17	Receipt dated after as-of	ignored in the calculation
18	Promise date on or after as-of	state Promised, not in the priority list
19	Promise date before as-of, still unpaid	state Collectable with note "Promise missed"
20	Negative or zero amount	row rejected with a message
21	Bad date, missing field, duplicate id	rejected with row and field named
22	Priority order with equal scores	days overdue, then collectable amount, then id
23	Explanation text for both worked examples in 5.8	matches the sentence exactly

#	Case	Expected
24	Draft uses the collectable amount and the gentle, firm or formal tone	matches Section 10
25	Two as-of dates on the same data	different buckets everywhere, no stored values
26	Promise 4000 made 2026-10-05 for 2026-10-12, as-of 2026-10-12, nothing received	state Promised, not ranked
27	Same promise, as-of 2026-10-13	state Collectable, broken promise, note "Promise missed on 2026-10-12, shortfall 4000"
28	Promise 3000, 1500 received after promise_made_date, promise date passed	shortfall 1500, BrokenPromise 0.5
29	Receipts of at least the promised amount received by the promise date	promise kept, BrokenPromise 0, no boost
30	Receipt dated before promise_made_date	not counted toward the promise
31	Promise recorded on a Paid invoice	ignored, state Paid
32	The three priority weights	sum to exactly 1.00
33	A: 45 days, 4000 collectable, broken promise 4000. B: 80 days, 1000 collectable, no promise. Largest collectable 8000	A scores 60.0, B scores 44.4, A ranks above B
34	Invoice 120 days overdue	AgeScore 1.0, same as 90 days
35	6000 outstanding with 2000 disputed (pending); and a single ranked invoice	AmountScore uses 4000, never 6000; a single ranked invoice gets AmountScore 1.0
36	Explanation text for the broken-promise example INV-2 in 5.8	matches the sentence exactly
37	Draft for a customer with a broken promise on a 1-30 day invoice	tone is at least Firm, names the promised date and the shortfall

#	Case	Expected
38	Promise with a missing field, a date before promise_made_date, or an amount above collectable	promise fields rejected, invoice kept
39	Two invoices with equal scores	order by days overdue, then collectable amount, then invoice id
40	Same data with as-of = promise date, then promise date + 1	Promised and not ranked, then Collectable and ranked higher because of the broken promise
41	Two receipts (3000 and 2000) on a 10000 invoice	outstanding 5000

The three case outcome checks from the problem statement are tests 1, 2 and 5 (with 4 and 25 for consistency). They must pass first.

13. Build order (follows the Fynd 10-stage flow)

Stage	What the AI and the team do
01 Research	Read the problem statement and fixture. List customer, pain and evidence. Note observed versus assumed in `docs/assumptions.md`
02 Design	Confirm the journey in Section 9, the in-scope list in Section 3, and the tests in Section 12
03 Technical plan	Confirm the layout (Section 7), rules (Section 5), storage (JSON in, JSON and CSV out) and failure handling (Section 6)
04 AI context	Save this document as `CONTEXT.md`. Add the real fixture field names and two sample rows

Stage	What the AI and the team do
05 Build	Step 1: `loader.py` with validation tests. Step 2: `rules.py` with tests 1-8 and 16-17. Step 3: `disputes.py` with tests 9-15 and 23. Step 4: promise and priority score, tests 18-22, 26-36 and 38-41. Step 5: `drafts.py`, tests 24 and 37. Step 6: `export.py`. Step 7: `app.py`. Commit to Git after each step
06 Test and release	Run all 41 tests. Try three as-of dates in the UI. Load a file with bad rows and confirm that nothing crashes. Write the evidence log. Tag a release
07 Business model	Engineering supplies cost-to-serve inputs only (hosting, maintenance hours), labelled as assumptions
08 First customers	Engineering supplies the demo and the export. Record who tried it, what they did, and their response
09 Fit or pivot	If the finance role player ignores or distrusts the explanation or the Review split, change the dispute fields or wording, and log the change
10 Grow	Propose the next 30-day measure: reconciliation accuracy (invoices matching expected balances out of total) and staff effort on a read-only sample

When the build reveals a mistake, update `CONTEXT.md`, rebuild the affected module, and retest (the "Iterate" loop in the flow).

14. Rules for the AI (do and do not)

Do

- Keep every business rule in `rules.py` or `disputes.py` as small pure functions.
- Pass `as_of` into every function. Never call `date.today()` in the rules.
- Use `Decimal` for money.
- Show clear messages for bad input. Never let the app crash.
- Keep the UI thin: it calls `evaluate_all` and displays the result.

- Write a test for every rule before or with the code.
- Keep the code short, readable and commented where a rule is not obvious.

Do not

- Do not add features, libraries or screens outside this document.
- Do not send messages or call external services.
- Do not auto-apply unapplied receipts.
- Do not store computed ageing; always recompute from the as-of date.
- Do not hide disputed balances. They must stay visible in the Review views.
- Do not claim recovered cash, live revenue or production readiness anywhere in the app or the docs.

15. Definition of done

- [ ] Fixture loads. Bad rows are reported, not fatal.
- [ ] 10000 with receipts 4000 shows 6000 outstanding.
- [ ] Paid invoices are absent from the priority list.
- [ ] Disputed balances are visible and held in Under Review.
- [ ] Changing the as-of date updates every bucket and ranking.
- [ ] Dispute form records cause and resolution (replace goods or credit), with the settled-amount explanation shown.
- [ ] Partial payment, unapplied receipt, disputed amount and promise-to-pay are all handled.
- [ ] A draft follow-up exists per customer and is marked as not sent.
- [ ] `result.json`, `result.csv` and `evidence_log.md` are produced.
- [ ] All 41 tests pass.
- [ ] `README.md` explains how to run the app and the tests in a few lines.

16. Feature checklist (master list of required features)

Every feature below comes from the problem statement or from the agreed focus (credit terms, disputes with cause and resolution, promise-to-pay, priority score). Each item says what to build, how to know it is done, and where the requirement comes from. Tick an item only when its tests pass. Use the IDs (F1, F2 and so on) in test names and commit messages.

A. Data in

- [ ] **F1 Import invoices and receipts.** Load invoices and receipts from `fixture.json` (or a CSV with the same fields) using `load_fixture`, mapping the real field names to the internal names. Done when: the fixture loads and the counts of invoices and receipts are shown. Source: Engineering build, "CSV or JSON handling".
- [ ] **F2 Data validation.** Apply every check in Section 6, including the promise checks, and show errors as a table (row, field, problem). Valid rows still load and the app never crashes on bad input. Done when: tests 14, 15, 20, 21 and 38 pass. Source: "data validation".
- [ ] **F3 Read-only source.** The input file is never changed. Edits (disputes, promises, review ticks) live in session state and in the exported result. Done when: the input file is identical after a full session. Source: "Path to a real pilot: read-only records".

B. Balances and ageing

- [ ] **F4 Outstanding balance.** `outstanding = amount - applied receipts - credit`, never below 0, with an Overpaid flag when it would be negative (Section 5.4). Done when: an invoice of 10000 with receipts of 4000 shows 6000 outstanding (tests 1 and 3). Source: case check, 10 points.
- [ ] **F5 Partial receipts.** A receipt smaller than the invoice reduces the balance by its amount only, and several receipts on one invoice add up. Done when: tests 1, 17 and 41 pass. Source: edge case "partial payment".
- [ ] **F6 Unapplied receipts.** A receipt with no invoice id is never subtracted from any invoice and never applied automatically. It is listed in its own tab, with a banner on that customer's invoices. Done when: test 16 passes. Source: edge case "unapplied receipt".
- [ ] **F7 Credit term, due date and overdue days.** Use the due date from the fixture, show `credit_days` when an invoice date exists, and compute `days_overdue = max(0, as_of - due_date)` (Section 5.2). Done when: tests 4, 6 and 7 pass. Source: agreed focus, "overdue calculation with clearly defined credit time".
- [ ] **F8 Ageing buckets.** Not due, 1-30, 31-60, 61-90 and 90+ (Section 5.3), recomputed on every as-of change. Done when: tests 5 and 8 pass. Source: case check "ageing".
- [ ] **F9 As-of date control.** A date picker in the sidebar. Changing it updates days overdue, buckets, states, scores, ranks, explanations and drafts everywhere, with nothing stored. Done when: tests 4, 5, 25 and 40 pass and a manual change in the UI updates every number. Source: case check, 5 points.

- [ ] **F10 Paid invoices leave the priority list.** An invoice with outstanding 0 is shown only in the Paid tab. Done when: tests 2 and 31 pass. Source: case check, 10 points.

C. Disputes

- [ ] **F11 Dispute record.** Capture the disputed amount and the cause (damaged goods, short delivery, wrong price, other) for an invoice, with the validation in Section 6. Done when: tests 14 and 15 pass and the form saves a valid dispute. Source: agreed focus, "dispute management and cause of dispute".
- [ ] **F12 Dispute resolution.** Record the outcome: pending, replace goods (delivered or not delivered), or credit note (financial settlement with a credit amount). Apply the table in Section 5.5. Done when: tests 9 to 13 pass. Source: agreed focus, "replace the goods or settle in finance".
- [ ] **F13 Settled-amount explanation.** Every invoice shows one sentence that states the amounts, the dispute, how it was resolved, what is settled, and what is still owed (Section 5.8). Done when: test 23 passes. Source: agreed focus, "clear explanation of the amount we settled".
- [ ] **F14 Disputed balances stay visible.** The disputed part is held in Under Review and never hidden. A partly disputed invoice shows its collectable part and its review part separately. Done when: tests 9 and 10 pass. Source: case check, 10 points, and edge case "disputed amount".

D. Promise to pay

- [ ] **F15 Promise record and hold.** Record `promise_made_date`, `promise_date` and `promise_amount`. While the promise is active, the invoice is Promised: not chased and not ranked. Done when: tests 18 and 26 pass. Source: edge case "promise-to-pay already received".
- [ ] **F16 Broken promise detection.** If the promise date has passed and a shortfall remains, the invoice returns to Collectable with a flag, the note "Promise missed on {promise_date}", and the shortfall (Section 5.6a). Done when: tests 19, 27 to 31 and 40 pass. Source: agreed focus, "customer agreed to pay on a date but did not pay".

E. Priority and follow-up

- [ ] **F17 Priority score and ranked list.** Score every Collectable invoice with the formula in Section 5.7 (weights 0.45, 0.35, 0.20), sort it, and show a "why ranked here" sentence on each row. Done when: tests 22 and 32 to 35 and 39 pass. Source: Engineering build,

"prioritize overdue invoices", and the agreed formula.

- [ ] **F18 Draft follow-up plan.** One draft per customer, with the tone set by ageing, review amounts and promise status (Section 10). The user can edit it and tick "Mark reviewed". It is always labelled "DRAFT - not sent" and is never sent. Done when: tests 24 and 37 pass. Source: Engineering build, "draft a reviewable follow-up plan".

F. UI, evidence and tests

- [ ] **F19 UI and state.** One Streamlit page as in Section 9: sidebar controls, top strip, the tabs, the invoice detail panel with the dispute and promise form. Every number comes from `evaluate_all`, and edits live in session state. Done when: a manual walk-through works (change the as-of date, save a dispute, save a promise, review a draft) with no errors. Source: capability "UI and state".
- [ ] **F20 Saved result.** The Export button writes `output/result.json` and `output/result.csv` with every computed field and the as-of date used. Done when: both files exist and reproduce the numbers on screen. Source: capability "readable evidence and a saved result".
- [ ] **F21 Audit view per invoice.** The detail panel shows the explanation sentence, the score breakdown and a small table (amount, receipts, credit, outstanding, review, collectable, promise status, state), so a reviewer can see why the invoice landed in its bucket. Done when: the panel is correct for a paid, disputed, promised and broken-promise invoice. Source: "readable evidence".
- [ ] **F22 Evidence log.** Save `output/starting_state.json` on first load, and write `output/evidence_log.md` with the four sections in Section 11 (starting state, negotiated scope, final evidence, observed versus assumed, with the priority weights listed as Assumed). Done when: the log is produced by the Export button. Source: "Product change to negotiate: save the starting state, negotiated scope and final evidence".
- [ ] **F23 Unit tests.** Every test in Section 12 (tests 1 to 41) is written in pytest and passes. Done when: `pytest` reports all 41 passing. Source: capability "tested business rules".
- [ ] **F24 Honest claims.** The app and the log state that the figures come from synthetic data, that drafts are not sent, and that no cash recovery or production readiness is claimed. Reconciliation accuracy and staff effort are measured by the team in a pilot on read-only records. The app only exports the data for that. Done when: the statement appears in the footer and in the evidence log. Source: "Path to a real pilot" and "Business proof required".

Not in this checklist: pricing, cost-to-serve, the first-customer path and the growth experiment. They are MBA deliverables.